CESDE

CESDE | comfama

Unidad I

Programación Orientada a Objetos (POO)

- **Conceptos base de POO:** clases, objetos, responsabilidades y modelado
- **Elementos de una clase:** atributos, métodos y constructores (sobrecarga básica)
- **Organización del código:** paquetes (packages), convenciones de nombres e importaciones
- **Modificadores de acceso:** public, private, protected, default y alcance
- **Encapsulamiento:** getters/setters, validaciones, invariantes y control de estado

Fundamentos de POO

Situación - El Cajero Automático "Roto"

Imagina que estás diseñando el software de un banco usando lo que sabes hasta ahora (variables sueltas y arrays). Tienes una variable double saldo = 1000;

En cualquier parte de tu código (o el de un compañero), alguien podría escribir saldo = -5000; o saldo = saldo * 0;

El sistema permite que el dinero desaparezca o sea negativo porque la variable está "desnuda" y desprotegida. No hay nadie vigilando que las reglas del banco se cumplan (ej: "No puedes retirar más de lo que tienes").

La Solución (POO)

En lugar de tener datos sueltos, creamos un **Agente Responsable** (un Objeto CuentaBancaria). Este agente tiene el dinero "dentro de su bolsillo" (**private**) y solo te deja tocarlo si le pides el favor educadamente a través de una ventanilla (**método public**), donde él verificará si la operación es válida. Si intentas poner un saldo negativo, el objeto te dirá: "Lo siento, eso viola mis reglas".

¿Qué es un paradigma de programación?

Un **paradigma de programación** es un **enfoque o estilo de programación** que determina **cómo se estructura y organiza un programa** para resolver un problema. (Arquitectura o plano general)

En otras palabras, es **una forma de pensar y escribir el código** que guía al programador sobre cómo deben crearse las soluciones.

Puedes hacer una pizza al horno (paradigma A), o puedes hacer una pizza frita (paradigma B). Ambos estilos producen pizza, pero el proceso y las herramientas que usas son diferentes.

Paradigma	En qué se basa	Ejemplo de lenguajes
Imperativo	Instrucciones paso a paso.	C, Pascal
Orientado a objetos	Objetos que combinan datos y acciones.	Java, C#, Python
Funcional	Funciones puras y sin efectos secundarios.	Haskell, Scala, JavaScript

Programación Orientada a Objetos (POO)

La **POO es un paradigma de programación** que organiza el código en *objetos*, que representan entidades del mundo real o abstracto. Cada objeto tiene propiedades (atributos) y acciones (métodos). Con unos pilares: **encapsulamiento, abstracción, herencia y polimorfismo.**

Modelar: es el arte de observar el mundo real y traducirlo a un diseño de software, quedándonos solo con lo importante.

Concepto	Definición	Ejemplo
Clase	Plantilla para crear objetos. Define atributos y métodos.	class Perro
Objeto	Instancia de una clase. Tiene valores concretos.	Perro rocky = new Perro();
Abstracción	Ignorar los detalles innecesarios y enfocarse en lo esencial.	Un Carro tiene encender(), frenar(), pero no detalles internos del motor.
Modularidad	Dividir el programa en partes que se pueden construir, probar y mantener de forma independiente.	Una clase para Usuario, otra para Producto.

- **Encapsulamiento (Protección de datos):** Oculta el estado interno de un objeto, restringiendo el acceso directo a sus datos (atributos) y permitiendo su modificación solo a través de métodos públicos seguros (métodos getter/setter).

- **Abstracción (Simplificación):** Se centra en las características esenciales de un objeto, ocultando detalles técnicos irrelevantes para el usuario. Define una interfaz clara y simple

- **Herencia (Reutilización):** Permite que una clase nueva (subclase o hija) herede atributos y métodos de una clase existente (superclase o padre), promoviendo la reutilización de código y jerarquías.

- **Polimorfismo (Flexibilidad):** Capacidad de diferentes objetos para responder de manera única a una misma interfaz o método. Permite tratar objetos de distintas clases como si fueran de una clase base común.

Clase vs. Objeto

Clase (El Plano/Molde): Es la definición abstracta. Define **qué** atributos tendrá y **qué** acciones podrá realizar. No ocupa memoria de ejecución para datos específicos. Es un contrato.

- *Ejemplo:* El plano arquitectónico de una casa.

Objeto (La Instancia): Es la materialización de la clase en la memoria RAM. Tiene su propio estado (valores en sus atributos).

- *Ejemplo:* La casa construida en la Calle 10 #20. Puedes tener 100 casas (objetos) del mismo plano (clase), pero cada una tiene dueños diferentes.

Analogía visual con objetos del mundo real

Clase: “Animal” → color, raza→ ruido(), correr().

Objeto: miAnimal= new Animal("Rojo", "Perro")

Responsabilidades

Un objeto no es solo una bolsa de datos (eso sería un struct o un registro en Excel). Un objeto en Java tiene **Responsabilidades**.

Los objetos deben ser dueños de sus propios datos y responsables de sus acciones.

- ***Mal diseño:*** El objeto Tienda le saca el dinero de la billetera al objeto Cliente. (La tienda manipula datos ajenos).
-
- ***Buen diseño (Responsabilidad):*** La Tienda le **pide** al Cliente que pague. El Cliente verifica si tiene saldo y reduce su propio dinero.

Atributos vs. Métodos

- **Atributos (Sustantivos):** Lo que el objeto **ES** o **TIENE**. (Color, Saldo, Nombre).
- **Métodos (Verbos):** Lo que el objeto **HACE** o **SABE HACER**. (Ladrar, CalcularImpuesto, Caminar).

Elementos de una Clase y Organización

Para que una clase sea funcional y profesional, debe tener una estructura definida.

Paquetes (package): Es la primera línea del código. Un paquete es un **Espacio de Nombres (Namespace)** que agrupa clases relacionadas lógicamente. Físicamente, un paquete es una **Carpeta** (directorio) en tu sistema operativo.

> ***Convención:*** Dominio invertido + nombre proyecto.
>
> Ej: co.edu.cesde.banco.modelo.

Importaciones (import)

Si la clase Main está en el paquete app y quiere usar la clase Empleado que está en el paquete modelo, no la puede ver directamente. Están en "habitaciones" diferentes.

El import es la instrucción que le dice al compilador: *"Oye, cuando escriba 'Empleado', me refiero al que está en la carpeta 'modelo'"*.

Atributos (Estado):

Son las variables que viven dentro de la clase. Representan las características.

```
public class Persona {
   String nombre;
   int edad;
}
```

Elemento	Convención	Ejemplo Correcto	Ejemplo Incorrecto
Clases	PascalCase (Primera mayúscula, sin guiones)	CuentaBancaria	cuentaBancaria, Cuenta_Bancaria
Métodos	camelCase (Primera minúscula, verbo)	calcularSalario()	CalcularSalario, calc_sal
Variables	camelCase (Primera minúscula, sustantivo)	saldoActual	Saldo, s
Paquetes	lowercase (Todo minúscula, separado por puntos)	co.edu.cesde.modelo	Co.Edu.Cesde, modeloClases
Constantes	UPPER_SNAKE_CASE (Mayúsculas con guion bajo)	VALOR_IVA	ValorIva, valor_iva

Para el encarpetado: La convención industrial estándar es usar los nombres de los paquetes en **inglés** y en **minúsculas**.

Constructores

Son métodos especiales que se ejecutan **una única vez** al momento de crear el objeto (new). Su misión es inicializar el objeto en un estado válido.

- Su **nombre debe ser igual al de la clase**.
- No tiene tipo de retorno, ni siquiera void.

```
public Persona(String nombre, int edad) {
    this.nombre = nombre;
    this.edad = edad;
}
```

Sobrecarga: Puedes tener múltiples constructores (uno vacío, otro con datos obligatorios) para ofrecer diferentes formas de crear el objeto.

Se Invocan con new: Los constructores se llaman implícitamente cuando creas una nueva instancia de una clase usando el operador new.

Persona mipersona = new Persona (); // Aquí se invoca el constructor de persona.

La Importancia de this en Constructores

En los constructores parametrizados (como public Persona (String nombre, int edad)), notarás el uso de this.nombre = nombre;

- this.nombre se refiere al **atributo de instancia** de la clase Producto.
- nombre (sin this.) se refiere al **parámetro** que se pasa al constructor.

this se utiliza para distinguir entre un atributo de la clase y un parámetro del constructor o método que tienen el mismo nombre. Es una buena práctica para mejorar la legibilidad y evitar ambigüedades.

Métodos (Comportamiento)

Es una acción que **pertenece** a un objeto y que, frecuentemente, **altera su estado** (sus atributos). Si los atributos son los sustantivos (Nombre, Saldo), los métodos son los verbos (Caminar, Pagar).

Para que Java entienda qué hace un método, debemos declarar su **Firma (Signature)**. La firma es lo que hace único al método dentro de la clase

```
//  1        2       3       4                5
  public    int    sumar   (int a, int b) { ... }
```

1. **Modificador de Acceso:** ¿Quién puede llamar a este método? (public, private).
2. **Tipo de Retorno:** ¿Qué entrega al final? (void si no entrega nada, o un tipo de dato int, String, Objeto).
3. **Nombre:** El verbo en CamelCase (calcularSalario).
4. **Parámetros:** La lista de ingredientes necesarios para trabajar.
5. **Cuerpo:** La lógica encapsulada.

Sobrecarga de Métodos (Overloading)

La sobrecarga es la capacidad de tener **múltiples métodos con el MISMO nombre** en la misma clase, siempre y cuando sus **parámetros sean diferentes**.

Las 3 Reglas de Oro de la Sobrecarga

Para que Java acepte dos métodos con el mismo nombre, debe variar **la lista de parámetros**:

1. **Cantidad diferente:** Uno recibe 1 parámetro, el otro recibe 2.
2. **Tipos diferentes:** Uno recibe un int, el otro recibe un String.
3. **Orden diferente:** Uno recibe (int, String), el otro (String, int).

ADVERTENCIA CRÍTICA:

Cambiar solo el **Tipo de Retorno** NO es sobrecarga. Si tienes public int suma(int a) y public double suma(int a), Java lanzará un error de compilación. La firma se basa en los parámetros, no en lo que devuelve.

PALABRAS CLAVE DEL MÉTODO

Tipos de Retorno (void) y Contexto de Memoria (static).

void: es un **Tipo de Retorno** especial que indica que el método ejecuta una acción pero no entrega ningún resultado al finalizar.

- **Método con Retorno (int, String):** El método ejecuta una acción y vuelve con un paquete que puede ser manejado por quien llamo al método.

- **Método void:** El método ejecuta una acción y listo, finaliza. **No regresa un paquete**. Hizo el trabajo, pero no devolvió un objeto.

static: El Recurso Compartido

Indica que el atributo o método pertenece a la **Clase** y no a una instancia específica. Se carga en memoria una sola vez al iniciar el programa.

- **No Estático (Instancia):** Es como un **Reloj de Pulsera**. Cada persona (Objeto) tiene el suyo. Si yo cambio la hora de mi reloj, no afecta al tuyo.
-
- **Estático (Clase):** Es como el **Reloj de Pared** del salón. Hay uno solo para todos. Si alguien lo adelanta 5 minutos, **todos** ven la hora cambiada

Modificadores de Acceso y Encapsulamiento

No es solo "ocultar datos". Es una técnica de ingeniería para proteger la integridad del objeto

Modificador	Alcance	Analogía
public	Todo el mundo (cualquier paquete).	La fachada de la casa.
protected	Mismo paquete + Hijos (Herencia).	El patio trasero (familia y vecinos).
default (sin palabra)	Solo el mismo paquete.	El pasillo del edificio.
private	Solo la propia clase.	Tu diario personal o caja fuerte.

Son el estándar industrial para leer y escribir datos.

- **Getter (getNombre):** Retorna el valor de un atributo. Permite controlar el formato de salida (ej: devolver el nombre siempre en mayúsculas).
- **Setter (setNombre):** Asigna un nuevo valor. Es el lugar crítico para las **Validaciones**.

Invariantes de Clase

Un **Invariante** es una regla de negocio que **siempre** debe ser verdadera para que el objeto sea válido.

- *Ejemplo:* "Un sueldo nunca puede ser negativo".
- *Ejemplo:* "Una fecha de nacimiento no puede ser futura".

Si permites usuario.sueldo = -500;, has roto el invariante. El objeto está "corrupto". El Encapsulamiento garantiza que el invariante nunca se rompa.

El **Setter** es el guardián. Su código (if, else, throw) es la **validación** que impide que entre un dato que rompa el **invariante**.

El **Getter** generalmente no valida nada, solo muestra

Preguntas Frecuentes

¿Por qué tengo que crear un constructor si Java ya me da uno por defecto?

Java te da un constructor vacío *solo si tú no escribes ninguno*. En el momento que defines un constructor con parámetros (obligando a que el objeto nazca con datos, como un documento de identidad), el constructor por defecto desaparece. Esto es bueno porque evitas tener objetos "vacíos" o inconsistentes.

¿Cuándo usar this?

this es una referencia al "objeto actual". Se usa principalmente para evitar la ambigüedad cuando un parámetro tiene el mismo nombre que un atributo (ej: this.nombre = nombre). Sin el this, estarías asignando la variable a sí misma y el atributo quedaría vacío.

Ejemplo Clase

```
public class Triangulo {
    private double base;
    private double altura;

    // CONSTRUCTOR
    // Responsabilidad: Asegurar el invariante al NACER.
    public Triangulo(double base, double altura) {
        // Si no validamos aquí, el objeto podría nacer "corrupto"
        setBase(base);
        setAltura(altura);
    }

    // SETTER (Mutator)
    // Responsabilidad: Asegurar el invariante al CAMBIAR.
    public void setBase(double base) {
        // Esta es la VALIDACIÓN que protege al INVARIANTE
        if (base > 0) {
            this.base = base;
        } else {
            System.out.println("Error: La base debe ser positiva. (Invariante protegido)");
        }
    }

    public void setAltura(double altura) {
        if (altura > 0) {
            this.altura = altura;
        } else {
            System.out.println("Error: La altura debe ser positiva.");
        }
    }
}
```

Demo Profesional

```java
package co.edu.cesde.sistema.modelo; // 1. Organización

public class Usuario { // Nombre en PascalCase

    // 2. Atributos (Estado) - SIEMPRE PRIVATE (casi siempre)
    private String documento;
    private String nombre;
    private int edad;

    // 3. Constructor (Inicialización)
    public Usuario(String documento, String nombre) {
        this.documento = documento;
        this.nombre = nombre;
        this.edad = 18; // Valor por defecto
    }

    // Sobrecarga de Constructor
    public Usuario(String documento, String nombre, int edad) {
        this.documento = documento;
        this.nombre = nombre;
        // Reutilizamos la lógica de validación del setter
        setEdad(edad);
    }

    // 4. Métodos de Acceso (Getters/Setters) con Lógica
    public String getNombre() {
        return nombre.toUpperCase(); // Podemos transformar al entregar
    }

    public void setEdad(int edad) {
        // 5. Validación de Invariante
        if (edad >= 0 && edad <= 120) {
            this.edad = edad;
        } else {
            System.out.println("Error: Edad no válida");
            // En avanzado lanzaríamos una Exception
        }
    }
}
```

Demo La Billetera Virtual

Vamos a implementar una clase que controle dinero, impida deudas (invariante) y registre transacciones básicas.

Paso 1: Definición de la clase y atributos privados.

¿Qué atributos debería tener una cuenta?

Paso 2: Constructor para saldo inicial.

Deberíamos crear el constructor con el saldo inicial

Paso 3: Métodos de negocio (ingresar, gastar) con validaciones.

Vamos a crear métodos para consignar, debidar, leer saldo y bloquear.

```java
public class Billetera {
    // Estado protegido
    private double saldo;
    private String propietario;
    private boolean estaBloqueada;

    // Constructor: Obligamos a definir dueño. Saldo arranca en 0.
    public Billetera(String propietario) {
        this.propietario = propietario;
        this.saldo = 0.0;
        this.estaBloqueada = false;
    }

    // Método para ingresar dinero (Setter disfrazado de lógica de negocio)
    public void recargar(double monto) {
        if (monto > 0) {
            this.saldo += monto;
            System.out.println("Recarga exitosa. Nuevo saldo: " + this.saldo);
        } else {
            System.out.println("Error: El monto a recargar debe ser positivo.");
        }
    }

    // Método complejo con validaciones múltiples
    public void pagar(double monto) {
        if (estaBloqueada) {
            System.out.println("Error: La billetera está bloqueada.");
            return; // Salir del método
        }

        if (monto > 0 && this.saldo >= monto) {
            this.saldo -= monto;
            System.out.println("Pago realizado de: " + monto + ". Quedan: " + this.saldo);
        } else {
            System.out.println("Error: Saldo insuficiente o monto inválido.");
        }
    }

    // Getter solo lectura (no hay setter para saldo, eso rompe el encapsulamiento)
    public double consultarSaldo() {
        return this.saldo;
    }

    // Setter para estado
    public void setBloqueo(boolean estado) {
        this.estaBloqueada = estado;
    }
}
```

Práctica Acompañada - Learning by Doing

Sistema de Control de Aire Acondicionado

Objetivo: Crear una clase que modele un aire acondicionado.

Requisitos:

1. Atributos: marca (String), temperatura (int), encendido (boolean).
2. La temperatura solo puede variar entre 16 y 30 grados.
3. Si está apagado, no se puede cambiar la temperatura.

Solución Paso a Paso:

1. Crear clase AireAcondicionado en paquete modelo.
2. Definir atributos private.
3. Constructor que recibe marca e inicializa en 24 grados y apagado.
4. Métodos encender() y apagar().
5. Métodos subirTemperatura() y bajarTemperatura() que validen si está encendido y los rangos (16-30).

Práctica Acompañada - La Calculadora Científica

Crear una clase Calculadora que pueda sumar de muchas formas

Paso a Paso:

Requisito 1: Crear el método sumar(int a, int b) que retorne un entero.

Requisito 2 (Sobrecarga): Crear sumar(double a, double b) para decimales.

Requisito 3 (Sobrecarga de cantidad): Crear sumar(int a, int b, int c) para tres números.

Requisito 4 (Sobrecarga de tipo): Crear sumar(int[] numeros) que reciba un array (lista básica) y sume todos sus elementos.

Pista requisito 4

```
public int sumar(int[] numeros) {
    int total = 0;
    for (int num : numeros) {
        total += num;
    }
    return total;
}
```

Troubleshooting (Errores Comunes)

NullPointerException: Crear la variable del objeto pero no inicializarla con new.

- *Error:* Billetera b; b.recargar(10);
- *Solución:* Siempre usar Billetera b = new Billetera("Dueño");

Shadowing (Sombreado): Olvidar el this en el constructor.

- *Error:* public Producto(String nombre) { nombre = nombre; } (Esto asigna la variable local a sí misma, el atributo queda null).
- *Solución:* this.nombre = nombre;

Acceso Directo: Intentar objeto.atributo cuando es privado desde el Main.

- *Solución:* Usar los métodos públicos (Getters/Setters/Negocio).

Ejercicio Independiente - Sistema de Reserva de Hotel Simple

Crea una clase llamada Habitacion.

1. **Atributos:** numero (int), tipo (String: "Sencilla", "Doble"), precioNoche (double), estaOcupada (boolean).
2. **Reglas (Invariantes):**
   - El número de habitación debe ser de 3 dígitos (ej: 101 a 999).
   - El precio no puede ser negativo.
3. **Funcionalidad:**
   - Un método reservar(): Solo funciona si NO está ocupada. Cambia el estado a ocupada.
   - Un método liberar(): Cambia el estado a libre.
   - Un método calcularTotal(int noches): Devuelve el precio * noches.

Criterio de Logro:

- Debes probar en un main intentar reservar una habitación ya ocupada (debe fallar).
- Intentar crear una habitación con número 5 (debe fallar o ajustarse).

¿Qué más debo saber?

- **Objetos Inmutables:** A veces creamos clases sin "Setters". Una vez creadas en el constructor, sus datos nunca cambian (como la clase String en Java). Esto es muy seguro para sistemas concurrentes.

- **Método toString():** Java permite que tus objetos se "presenten". Si escribes un método public String toString() en tu clase, cuando hagas System.out.println(miObjeto), no saldrá un código raro de memoria, sino el texto que tú definas ahí.

CESDE | comfama